UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final

Disciplina: Qualidade de Software
Equipe: Maria Letícia dos Santos Dantas - 555697 e João Vitor Rodrigues Laurindo [matrícula]
Projeto: NVIDIA NeMo Guardrails (https://github.com/NVIDIA-NeMo/Guardrails)
Fork: https://github.com/letdnts/Guardrails (branch refatoracao-test-smells)
Pull Request: https://github.com/NVIDIA-NeMo/Guardrails/pull/2537
Commit SHA: fe6a9c01184695a6b84c02fd2efe6dd3a607575c

1. Apresentação do projeto

O projeto escolhido foi o **NVIDIA NeMo Guardrails**, uma biblioteca de código aberto desenvolvida principalmente em Python. Ela permite adicionar regras de comportamento e segurança a aplicações que utilizam grandes modelos de linguagem (LLMs), como chatbots e assistentes virtuais. O projeto atende desenvolvedores e equipes que precisam controlar as interações dessas aplicações, por exemplo, limitando assuntos abordados, moderando conteúdos e definindo fluxos de diálogo.

Os testes são organizados principalmente no diretório tests/, com arquivos e subdiretórios relacionados às funcionalidades avaliadas, como cache, configurações, integrações e interface de linha de comando. O diretório benchmark/tests/ reúne testes das ferramentas de benchmark. A execução utiliza o **pytest**, incluindo testes escritos como funções e como classes do unittest. Também são utilizados fixtures e mocks para preparar cenários e simular dependências de forma controlada.

2. Estado inicial do projeto

Test smell	Ocorrências detectadas
Unknown Test	593
Conditional Test Logic	369
Verbose Test	116
Programming Paradigms Blend	97
Non-Functional Statement	46

Test smell	Ocorrências detectadas
Exception Handling	45
Undefined Test	25
Sleepy Test	15
Redundant Print	6
Total	**1.312**

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	7.228
Testes com falha (Failed)	0
Erros (Errors)	0
Ignorados (Skipped)	214
Cobertura total (%)	86,51

Cobertura total com ramos (pytest-cov): 25.241 linhas cobertas; o relatório HTML a exibe arredondada para 87 %. Medição no commit fe6a9c0.

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	104.385
Complexidade ciclomática média das funções de teste	2,881
Índice de manutenibilidade médio dos arquivos de teste	62,520

Considerando apenas linhas com origem = teste: 7.699 funções e métodos (CC média) e 476 arquivos (MI médio). Se a CC média fosse calculada só sobre as 5.992 funções cujo nome começa com test, o valor seria 3,176.

3. Refatorações realizadas

3.1 Seleção das ocorrências

Escolhemos quatro tipos de test smell: Conditional Test Logic, Verbose Test, Sleepy Test e Redundant Print. Os quatro são detectados pelo TEMPY no projeto (369, 116, 15 e 6 ocorrências, respectivamente) e têm página no CATSPY com definição, efeitos negativos e orientação de refatoração, o que nos permitiu justificar cada decisão com o catálogo. São também smells que se corrigem apenas no código de teste, sem alterar o código de produção, e que exigem estratégias diferentes: parametrização e divisão de testes (Conditional Test

Logic), extração de funções auxiliares, fábricas e fixtures (Verbose Test), relógio simulado e semáforos no lugar de esperas explícitas (Sleepy Test) e remoção direta (Redundant Print).

Dentro de cada tipo, a dupla seguiu um roteiro de refatoração com um commit por refatoração, dividido entre os integrantes (Seção 7). Ao todo foram 43 ocorrências, acima das 40 exigidas: 23 de Conditional Test Logic, 10 de Verbose Test, 8 de Sleepy Test e 2 de Redundant Print.

3.2 Lista das refatorações

As ocorrências seguem a ordem do diário de bordo (registros 01 a 16 de João Vitor e 17 a 35 de Letícia). Registros com mais de uma ocorrência ocupam mais de uma linha, com o mesmo commit. Os hashes de João Vitor foram copiados do diário dele.

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Sleepy Test	tests/test_cache_lfu.py::TestLFUCacheStatsLogging::test_stats_logging_with_empty_cache	e652354	sim	sim	não
R02	Sleepy Test	tests/test_cache_lfu.py::TestLFUCacheStatsLogging::test_periodic_stats_logging	e824f5f	sim	sim	não
R03	Sleepy Test	tests/test_cache_lfu.py::TestLFUCacheStatsLogging::test_periodic_stats_logging	e824f5f	sim	sim	não
R04	Sleepy Test	tests/test_cache_lfu.py::TestLFUCacheStatsLogging::test_stats_logging_with_full_cache	c255e77	sim	sim	não
R05	Verbose Test	tests/test_cache_lfu.py::TestLFUCache::test_complex_eviction_scenario	991f934	sim	sim	não
R06	Verbose Test	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_reads_writes	9cf2e4a	sim	sim	não
R07	Verbose Test	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_get_or_compute_thread_safety	81263cd	sim	sim	não
R08	Verbose Test	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_operations_with_evictions	96013d2	sim	sim	não
R09	Verbose Test	benchmark/tests/test_mock_response_data.py::TestGenerateChunkLat	2b59c67	sim (77 testes,	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		encies::test_generate_latencies_multiple_chunks_no_seed		segundo o diário)		
R10	Redundant Print	tests/cli/test_migration.py::TestColang1SyntaxConversion::test_complex_conversion	df14b83	sim (teste já ignorado)	sim	não
R11	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_thread_safe_size_operations	89214de	sim	sim	não
R12	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_thread_safe_size_operations	89214de	sim	sim	não
R13	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_contains_operations	8ee7a6a	sim	sim	não
R14	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_contains_operations	8ee7a6a	sim	sim	não
R15	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_contains_operations	8ee7a6a	sim	sim	não
R16	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCache::test_repeated_puts_same_key	6d4741d	sim	sim	não
R17	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_stats_operations	088d217	sim	sim	não
R18	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_reset_stats	a2121b7	sim	sim	não
R19	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheStatsLogging::test_stats_log_format_percentages	a90a557	sim	sim	não
R20	Conditional Test Logic	tests/test_cache_lfu.py::TestLFUCacheStatsLogging::test_stats_log_format_percentages	7dfbb38	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R21	Redundant Print	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_contains_operations	330097f	sim	sim	não
R22	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestCohereEmbeddingModelMocked::test_all_predefined_models	102596e	sim	sim	sim: Programming Paradigms Blend
R23	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestOpenAIEmbeddingModelMocked::test_all_predefined_models	6dfe48f	sim	sim	não
R24	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestCohereEmbeddingModelMocked::test_import_error_when_cohere_not_installed	c2c4065	sim	sim	não
R25	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestOpenAIEmbeddingModelMocked::test_import_error_when_openai_not_installed	0e9b833	sim	sim	não
R26	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestAzureEmbeddingModelMocked::test_import_error_when_openai_not_installed	1707914	sim	sim	não
R27	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestGoogleEmbeddingModelMocked::test_import_error_when_google_genai_not_installed	44fbf3a	sim	sim	não
R28	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestAzureEmbeddingModelMocked::test_all_predefined_models	c8b6ce1	sim	sim	não
R29	Conditional Test Logic	tests/test_embeddings_providers_mock.py::TestGoogleEmbeddingModelMocked::test_all_predefined_models	3fa9c6a	sim	sim	não
R30	Conditional Test Logic	tests/test_generation_options.py::test_generation_response_with_rail_outcome_is_json_serializable	5063181	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R31	Conditional Test Logic	tests/test_generation_options.py::test_generation_response_with_rail_outcome_is_json_serializable	5063181	sim	sim	não
R32	Conditional Test Logic	tests/test_generation_options.py::test_generation_log_with_rail_outcome_is_json_serializable	926b286	sim	sim	não
R33	Sleepy Test	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_clear_operations	1d01749	sim	sim	não
R34	Sleepy Test	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_clear_operations	1d01749	sim	sim	não
R35	Sleepy Test	tests/test_cache_lfu.py::TestLFUCacheThreadSafety::test_concurrent_clear_operations	1d01749	sim	sim	não
R36	Sleepy Test	tests/test_hf_classifier.py::TestPipelineConcurrency::test_concurrent_callers_wait_for_first_load	2dc3908	sim	sim	não
R37	Verbose Test	tests/test_llmrails.py::test_register_methods_return_self	b2b5fa7	sim	sim	não
R38	Verbose Test	tests/test_clavata_models.py::TestCreateJobResponse::test_model_validate_valid_json	0e6c88e	sim	sim	não
R39	Verbose Test	tests/test_clavata_models.py::TestCreateJobResponse::test_model_validate_complete_job_response	1f60a9a	sim	sim	não
R40	Verbose Test	tests/evaluate/test_evaluate_cli_and_data.py::test_topical_command_invokes_evaluation_class	2dabd22	sim	sim	não
R41	Conditional Test Logic	tests/test_action_dispatcher.py::test_load_all_actions_registers_builtin_library_action_refs_lazily	95fd210	sim	sim	não
R42	Conditional Test Logic	tests/test_action_dispatcher.py::test_load_all_actions_registers_builtin_library_action_refs_lazily	95fd210	sim	sim	não
R43	Verbose Test	tests/test_patronus_evaluate_api.py::test_patronus_evaluate_api_success_strategy_all_pass	2c217fb	sim	sim	não

3.3 Exemplos detalhados

R10 – Redundant Print (tests/cli/test_migration.py::TestColang1SyntaxConversion::test_complex_conversion)

Antes:

```
239     @pytest.mark.skip("not implemented conversion")
240     def test_complex_conversion(self):
241         # TODO: add bot $response to bot say $response conversion
242         input_script = """
243         define flow greeting_flow
244             when user express greeting
245                 $response = execute generate_greeting
246                 bot $response
247         """
248         expected_output_script = """
249         flow greeting_flow
250             when user express greeting
251                 $response = await GenerateGreetingAction
252                 bot say $response
253         """
254         input_lines = textwrap.dedent(input_script).strip().split("\n")
255         expected_output = textwrap.dedent(expected_output_script).strip().split("\n")
256 
257         print(convert_colang_1_syntax(input_lines))
258         assert convert_colang_1_syntax(input_lines) == expected_output
```

Depois:

```
239     @pytest.mark.skip("not implemented conversion")
240     def test_complex_conversion(self):
241         # TODO: add bot $response to bot say $response conversion
242         input_script = """
243         define flow greeting_flow
244             when user express greeting
245                 $response = execute generate_greeting
246                 bot $response
247         """
248         expected_output_script = """
249         flow greeting_flow
250             when user express greeting
251                 $response = await GenerateGreetingAction
252                 bot say $response
253         """
254         input_lines = textwrap.dedent(input_script).strip().split("\n")
255         expected_output = textwrap.dedent(expected_output_script).strip().split("\n")
256 
257         assert convert_colang_1_syntax(input_lines) == expected_output
```

Por que era um test smell: O teste chamava print(convert_colang_1_syntax(input_lines)) antes da asserção. Em uma suíte automatizada ninguém lê essa saída, e o print repete uma conversão que a asserção seguinte já executa. Além de poluir a saída do pytest, é uma instrução que não verifica nada.

Estratégia do CATSPY aplicada: Seguimos a orientação do catálogo de remover impressões desnecessárias e usar asserções para avaliar o resultado. Não foi preciso adaptar a estratégia: o teste já tinha a asserção que compara a conversão com o resultado esperado, então bastou remover o print. O teste já estava marcado com skip porque a conversão ainda não está implementada, e essa condição foi mantida.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	544	543

A complexidade ciclomática não muda porque o print não tem ramificação; o arquivo perde uma linha de código.

R22 – Conditional Test Logic (tests/test_embeddings_providers_mock.py::TestCohereEmbeddingModelMocked::test_all_predefined_models)

Antes:

```
168    def test_all_predefined_models(self):
169        mock_cohere = MagicMock()
170        mock_client = Mock()
171        mock_cohere.Client.return_value = mock_client
172
173        models_to_test = {
174            "embed-v4.0": 1536,
175            "embed-english-v3.0": 1024,
176            "embed-english-light-v3.0": 384,
177            "embed-multilingual-v3.0": 1024,
178            "embed-multilingual-light-v3.0": 384,
179        }
180
181        with patch.dict("sys.modules", {"cohere": mock_cohere}):
182            from nemoguardrails.embeddings.providers.cohere import CohereEmbeddingModel
183
184            for model_name, expected_size in models_to_test.items():
185                model = CohereEmbeddingModel(model_name)
186                assert model.embedding_size == expected_size
187                assert model.model == model_name
```

Depois:

```
COHERE_MODELS = [
    ("embed-v4.0", 1536),
    ("embed-english-v3.0", 1024),
    ("embed-english-light-v3.0", 384),
    ("embed-multilingual-v3.0", 1024),
    ("embed-multilingual-light-v3.0", 384),
]


# ... (class TestCohereEmbeddingModelMocked) ...

    @pytest.mark.parametrize("model_name, expected_size", COHERE_MODELS)
    def test_all_predefined_models(self, model_name, expected_size):
        mock_cohere = MagicMock()
        mock_client = Mock()
        mock_cohere.Client.return_value = mock_client

        with patch.dict("sys.modules", {"cohere": mock_cohere}):
            from nemoguardrails.embeddings.providers.cohere import CohereEmbeddingModel

            model = CohereEmbeddingModel(model_name)
            assert model.embedding_size == expected_size
            assert model.model == model_name
```

Por que era um test smell: O teste usava um for sobre o dicionário models_to_test para verificar cinco modelos da Cohere em um único teste. Com o laço, a primeira falha interrompe a

verificação dos modelos seguintes e o pytest não mostra cada modelo como um caso independente.

Estratégia do CATSPY aplicada: A definição do Conditional Test Logic inclui laços como o for entre as estruturas de controle que não devem ficar no corpo do teste. Aplicamos a orientação de tornar explícito o cenário esperado, adaptando-a ao pytest: cada iteração virou um caso com @pytest.mark.parametrize e a lista COHERE_MODELS. O mock de cohere e o patch.dict continuam dentro do teste, e as asserções de embedding_size e model foram mantidas.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	4	3
SLOC do arquivo de teste (raw_por_arquivo_e_total)	635	631

O SLOC do arquivo inclui também as outras sete refatorações feitas em tests/test_embeddings_providers_mock.py.

R36 – Sleepy Test (tests/test_hf_classifier.py::TestPipelineConcurrency::test_concurrent_callers_wait_for_first_load)

Antes:

```
868    def test_concurrent_callers_wait_for_first_load(self):
869        """A second caller seeing _PipelineLoading should wait for the first
870        loader to finish and receive the same pipeline."""
871        key = _pipeline_cache_key("shared-model", "text-classification", {})
872        load_started = threading.Event()
873        mock_pipe = mock.MagicMock()
874
875        def slow_pipeline(**kwargs):
876            load_started.set()
877            time.sleep(0.1)
878            return mock_pipe
879
880        mock_transformers = mock.MagicMock()
881        mock_transformers.pipeline = slow_pipeline
882
883        results = [None, None]
884
885        def loader(idx):
886            results[idx] = _get_or_create_pipeline("shared-model", "text-classification", {})
887
888        with mock.patch.dict("sys.modules", {"transformers": mock_transformers}):
889            t1 = threading.Thread(target=loader, args=(0,))
890            t1.start()
891            load_started.wait(timeout=5)
892
893            t2 = threading.Thread(target=loader, args=(1,))
894            t2.start()
895
896            t1.join(timeout=5)
897            t2.join(timeout=5)
898
899        assert results[0] is mock_pipe
900        assert results[1] is mock_pipe
```

Depois:

```
868     def test_concurrent_callers_wait_for_first_load(self):
869         """A second caller seeing _PipelineLoading should wait for the first
870         loader to finish and receive the same pipeline."""
871         key = _pipeline_cache_key("shared-model", "text-classification", {})
872         load_started = threading.Event()
873         mock_pipe = mock.MagicMock()
874 
875         release_load = threading.Semaphore(0)
876 
877         def slow_pipeline(**kwargs):
878             load_started.set()
879             assert release_load.acquire(timeout=5)
880             return mock_pipe
881 
882         mock_transformers = mock.MagicMock()
883         mock_transformers.pipeline = slow_pipeline
884 
885         results = [None, None]
886 
887         def loader(idx):
888             results[idx] = _get_or_create_pipeline("shared-model", "text-classification", {})
889 
890         with mock.patch.dict("sys.modules", {"transformers": mock_transformers}):
891             t1 = threading.Thread(target=loader, args=(0,))
892             t1.start()
893             load_started.wait(timeout=5)
894 
895             t2 = threading.Thread(target=loader, args=(1,))
896             t2.start()
897 
898             release_load.release()
899             t1.join(timeout=5)
900             t2.join(timeout=5)
901 
902         assert results[0] is mock_pipe
903         assert results[1] is mock_pipe
```

Por que era um test smell: A função slow_pipeline usava time.sleep(0.1) para simular um carregamento lento, de modo que o segundo chamador chegasse enquanto o primeiro ainda carregava. Com tempo fixo, essa sobreposição era apenas esperada, e o teste dependia da velocidade da máquina.

Estratégia do CATSPY aplicada: O catálogo recomenda evitar esperas explícitas e substituí-las por mecanismos controlados, com o uso de mocks como alternativa. Um relógio simulado, que usamos nos testes de logging, não resolve aqui, porque o que importa é a ordem entre as threads. Adaptamos a estratégia para um threading.Semaphore: slow_pipeline só retorna quando o teste libera o semáforo, depois de iniciar a segunda thread.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	3	3
SLOC do arquivo de teste (raw_por_arquivo_e_total)	806	808

O SLOC aumenta duas linhas (o semáforo e a liberação), um custo pequeno para remover a dependência de tempo.

R40 – Verbose Test (tests/evaluate/test_evaluate_cli_and_data.py::test_topical_command_invokes_evaluation_class)

Antes:

```python
def test_topical_command_invokes_evaluation_class(tmp_path):
    config = tmp_path / "config"
    output = tmp_path / "output"
    config.mkdir()
    topical_eval = MagicMock()

    with (
        patch("nemoguardrails.evaluate.cli.evaluate.TopicalRailsEvaluation", return_value=topical_eval) as mock_cls,
        patch("nemoguardrails.evaluate.cli.evaluate.set_verbose") as mock_set_verbose,
    ):
        result = runner.invoke(
            evaluate.app,
            [
                "topical",
                "--config",
                str(config),
                "--verbose",
                "--test-percentage",
                "0.4",
                "--max-tests-intent",
                "4",
                "--max-samples-intent",
                "2",
                "--results-frequency",
                "5",
                "--sim-threshold",
                "0.75",
                "--random-seed",
                "123",
                "--output-dir",
                str(output),
            ],
        )

    assert result.exit_code == 0
    mock_set_verbose.assert_called_once_with(True)
    mock_cls.assert_called_once_with(
        config=str(config),
        verbose=True,
        test_set_percentage=0.4,
        max_samples_per_intent=2,
        max_tests_per_intent=4,
        print_test_results_frequency=5,
        similarity_threshold=0.75,
        random_seed=123,
        output_dir=str(output),
    )
    topical_eval.evaluate_topical_rails.assert_called_once()
```

Depois:

```python
def _topical_cli_args(config, output):
    return [
        "topical",
        "--config",
        str(config),
        "--verbose",
        "--test-percentage",
        "0.4",
        "--max-tests-intent",
        "4",
        "--max-samples-intent",
        "2",
        "--results-frequency",
        "5",
        "--sim-threshold",
        "0.75",
        "--random-seed",
        "123",
        "--output-dir",
        str(output),
    ]


def test_topical_command_invokes_evaluation_class(tmp_path):
    config = tmp_path / "config"
    output = tmp_path / "output"
    config.mkdir()
    topical_eval = MagicMock()

    with (
        patch("nemoguardrails.evaluate.cli.evaluate.TopicalRailsEvaluation", return_value=topical_eval) as mock_cls,
        patch("nemoguardrails.evaluate.cli.evaluate.set_verbose") as mock_set_verbose,
    ):
        result = runner.invoke(evaluate.app, _topical_cli_args(config, output))

    assert result.exit_code == 0
    mock_set_verbose.assert_called_once_with(True)
    mock_cls.assert_called_once_with(
        config=str(config),
        verbose=True,
        test_set_percentage=0.4,
        max_samples_per_intent=2,
        max_tests_per_intent=4,
        print_test_results_frequency=5,
        similarity_threshold=0.75,
        random_seed=123,
        output_dir=str(output),
    )
    topical_eval.evaluate_topical_rails.assert_called_once()
```

Por que era um test smell: O teste tinha 48 linhas, e a lista de argumentos da linha de comando passada ao runner.invoke ocupava grande parte delas, escondendo o que o teste de fato verifica (o código de saída e os argumentos repassados à classe de avaliação).

Estratégia do CATSPY aplicada: Seguimos a recomendação do Verbose Test de reduzir o tamanho do teste e separar a construção dos dados do que é verificado. Adaptamos para não esconder a intenção: extraímos apenas a lista para a função _topical_cli_args e mantivemos no corpo do teste o runner.invoke e todas as asserções (exit_code, set_verbose, os argumentos da classe de avaliação e a chamada a evaluate_topical_rails).

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	272	271

A complexidade do teste continua 2; a nova função auxiliar _topical_cli_args tem complexidade 1. O alerta de Verbose Test (linhas 52–99) deixou de ser apontado.

4. Estado final e comparação

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	1.312	1.260	−52
Testes aprovados	7.228	7.241	+13
Testes com falha	0	0	0
Cobertura total (%)	86,51	86,51	0,00 p.p.
SLOC dos testes	104.385	104.360	−25
Complexidade ciclomática média das funções de teste	2,881	2,872	−0,009
Índice de manutenibilidade médio dos arquivos de teste	62,5203	62,5188	−0,0015

Test smell	Antes	Depois	Variação
Unknown Test	593	593	0
Conditional Test Logic	369	340	−29
Verbose Test	116	105	−11
Programming Paradigms Blend	97	98	+1
Non-Functional Statement	46	43	−3
Exception Handling	45	45	0
Undefined Test	25	25	0
Sleepy Test	15	7	−8
Redundant Print	6	4	−2
Total	**1.312**	**1.260**	**−52**

Medição inicial no commit fe6a9c0 e medição final no commit 2c217fb (ponta da branch refatoracao-test-smells). Os totais do projeto refletem o trabalho dos dois integrantes. Cobertura: 25.241 linhas cobertas nas duas medições.

5. Análise dos resultados

Quais métricas melhoraram e quais pioraram?

Melhoraram: o total de test smells caiu de 1.312 para 1.260 (−52, cerca de 4 %): Conditional Test Logic −29, Verbose Test −11, Sleepy Test −8, Non-Functional Statement −3 e Redundant Print −2. O SLOC dos testes caiu 25 linhas (104.385 para 104.360) e a complexidade ciclomática média passou de 2,881 para 2,872.

A queda de Conditional Test Logic (−29) e de Verbose Test (−11) é maior que as 23 e as 10 ocorrências que declaramos, porque remover uma condição, um laço ou dividir um teste longo costuma eliminar alertas vizinhos que o TEMPY também contava (por exemplo, as refatorações de Verbose Test dos testes de cache eliminaram condicionais dentro deles). O mesmo vale para os três Non-Functional Statement, que sumiram junto com a refatoração de dois testes de logging e do teste do Patronus, sem terem sido alvo direto.

Pioraram: Programming Paradigms Blend aumentou em 1 (97 para 98), tratado na próxima pergunta, e o índice de manutenibilidade médio caiu 0,0015 (62,5203 para 62,5188), variação desprezível.

Alguma refatoração introduziu novos test smells? Quais?

Sim, um: Programming Paradigms Blend, em tests/test_embeddings_providers_mock.py, em um alerta no nível do módulo (linhas 22 e 31 na medição final). Essas linhas correspondem à lista COHERE_MODELS, que declaramos acima da classe de testes da Cohere ao parametrizar o teste (Registro 18), e à própria classe; por isso atribuímos o alerta a essa refatoração. As listas OPENAI_MODELS, AZURE_MODELS e GOOGLE_MODELS não geraram alertas adicionais.

Nenhum outro smell novo apareceu. Comparamos, nos arquivos smells.csv de antes e depois, as combinações de arquivo, tipo e método e todas as demais diferenças são reduções. No Registro 28, escolhemos uma solução sem if justamente para não criar um Conditional Test Logic novo: o total desse smell no método ficou em 5 antes e depois.

O resultado dos testes ou a cobertura mudou?

O resultado dos testes não piorou: foram 7.228 testes aprovados antes e 7.241 depois, com 0 falhas e 0 erros nas duas medições e os mesmos 214 testes ignorados. A cobertura total ficou em 86,51 % nas duas medições, com as mesmas 25.241 linhas cobertas.

O aumento de 13 testes vem apenas de casos novos criados pelas refatorações: +6 da parametrização dos testes de embeddings (Cohere +4 e OpenAI +2), +6 da divisão de test_register_methods_return_self em sete testes e +1 da divisão de test_complex_eviction_scenario em dois. Os 18 casos de test_generation_options foram substituídos por 18 casos com novos nomes. Os testes de benchmark/tests não aparecem nos relatórios de pytest das métricas; para eles usamos a validação registrada no diário (77 testes aprovados).

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Mais fáceis: Redundant Print (R10 e R21), que consistiu em remover uma linha sem tocar nas asserções, e os Conditional Test Logic baseados em padrões repetíveis, como a parametrização dos testes de modelos (R22, R23, R28 e R29) e o uso de sys.modules.pop nos testes de ImportError (R24 a R27).

Mais difíceis: Sleepy Test em testes de concorrência (R33 a R36). Trocar a espera por sincronização exige entender a ordem das threads para continuar testando a mesma coisa; nos testes de logging, foi preciso controlar o relógio do cache para manter as verificações do intervalo. Verbose Test em testes de threads (R06 a R08) também deu trabalho, porque as funções auxiliares extraídas precisavam preservar o número de threads e operações e a

propagação de falhas por future.result(). Entre os Conditional Test Logic, o R16 exigiu entender o cenário: a asserção sobre estatísticas nunca executava porque a configuração padrão do setUp desativa track_stats, e a condição escondia isso.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Usamos o CATSPY como referência antes de cada refatoração: consultávamos a página do tipo de smell (Conditional Test Logic, Verbose Test, Sleepy Test ou Redundant Print) para confirmar a definição, entender os efeitos negativos e escolher a estratégia. Cada registro do diário traz o link da página consultada e a decisão tomada a partir dela. Também usamos o catálogo para não trocar um smell por outro: no Registro 28, por exemplo, descartamos a versão com if porque ela criaria um novo Conditional Test Logic.

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

A definição e a orientação de refatoração foram as partes mais úteis, porque diziam o que fazer: substituir esperas explícitas por mecanismos controlados, dividir testes longos em testes menores e focados, remover impressões e usar asserções, e explicitar o cenário esperado em vez de condicioná-lo. Os efeitos negativos ajudaram a justificar cada mudança nos registros e neste relatório.

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim, em alguns casos foi preciso adaptar. Em Sleepy Test, o catálogo apresenta o mock como alternativa, o que funcionou nos testes de logging (relógio simulado), mas não nos testes de concorrência, em que o problema é a ordem entre threads; nesses usamos threading.Semaphore (R33 a R36). Em Conditional Test Logic, a orientação de explicitar o cenário foi traduzida para o pytest com @pytest.mark.parametrize e sys.modules.pop. Em Verbose Test, dividir o teste não bastava em alguns casos: usamos fábricas e funções auxiliares (R38 a R40 e R43) para reduzir o tamanho sem esconder o que o teste verifica.

O que faltou ou poderia ser melhorado no catálogo?

Sugerimos: (1) exemplos de refatoração de Sleepy Test em testes com threads, usando semáforos ou eventos, além de mocks de relógio; (2) exemplos com @pytest.mark.parametrize para Conditional Test Logic, já que muitos projetos usam pytest; e (3) um aviso de que extrair constantes ou funções auxiliares para o nível do módulo pode disparar Programming Paradigms Blend, como aconteceu no Registro 18.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
João Vitor Rodrigues Laurindo	R01 a R20	20

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Maria Letícia dos Santos Dantas	R21 a R43	23

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

(1) Preservar o comportamento em testes de concorrência: ao remover sleeps e condições dentro de threads, era preciso manter as mesmas verificações e garantir que as falhas continuassem chegando ao teste principal (future.result()).

(2) Entender o que cada alerta do TEMPY conta: laços também são Conditional Test Logic, alguns alertas (como Unknown Test) permaneceram depois das refatorações, e uma constante no nível do módulo gerou um alerta novo (Programming Paradigms Blend).

(3) Seguir o roteiro sem copiá-lo cegamente: no Registro 28, a solução prevista criaria um if no writer e um novo Conditional Test Logic, então refizemos de outra forma.

(4) Evitar trabalho duplicado: o João Vitor já havia refatorado test_generate_latencies_multiple_chunks_no_seed (Registro 08), então esse item do roteiro não foi refeito pela Letícia.

(5) Tempo de execução: a suíte completa leva cerca de 30 minutos (32 min 36 s na medição inicial e 26 min 44 s na final). Por isso validamos cada commit com os testes específicos e reservamos a suíte completa para as medições.

O que você aprendeu com a atividade?

Aprendemos que um test smell raramente é só estético: o if que impedia uma asserção de executar (R16) e os sleeps que tornavam os testes dependentes da máquina eram riscos reais de confiabilidade. Aprendemos também que a sincronização explícita (semáforos, relógio simulado) deixa os testes de concorrência mais determinísticos que esperas fixas, que as métricas de código mudam pouco quando as refatorações apenas redistribuem linhas (SLOC −25 em mais de 104 mil), embora os alertas e a legibilidade melhorem, e que é preciso medir antes e depois, porque uma refatoração pode introduzir outro smell sem que se perceba.

9. Contribuição (Pull Request)

O PR reúne os commits da branch refatoracao-test-smells do fork letdnts/Guardrails, um por refatoração (35 commits: 16 de João Vitor e 19 de Letícia), com mensagens no padrão refactor(tests): … e cobrindo as 43 ocorrências da Seção 3.2. 10. Observações gerais

Numeração: os IDs da Seção 3.2 seguem a ordem do diário de bordo (João Vitor primeiro, depois Letícia), e não a ordem exata dos commits no tempo, já que os dois trabalharam em paralelo entre 25 e 27/10/2026.

Total: a dupla refatorou 43 ocorrências (20 de João Vitor e 23 de Letícia), acima das 40 exigidas. Letícia tinha 20 ocorrências previstas no roteiro; deixou de fazer uma (o Registro 08 de João Vitor já cobria o mesmo teste) e acrescentou quatro (um segundo alerta no Registro 26, duas ocorrências no Registro 34 e uma no Registro 35), chegando a 23.

Fora de escopo: os 593 alertas de Unknown Test e os demais tipos não escolhidos não foram tratados. Em testes refatorados, alguns Unknown Test já existentes continuaram sendo apontados (Registros 20, 22, 23 e 35 do diário, correspondentes às ocorrências R24, R26, R27 e R43).

Medições: o “antes” foi medido no commit fe6a9c0 e o “depois” no commit 2c217fb, sem alterar o ambiente. A complexidade ciclomática média usa todas as funções e métodos de arquivos de teste (origem = teste). As imagens da Seção 3.3 foram geradas a partir do código do projeto antes e depois da refatoração, com as linhas alteradas destacadas.